基于紫光同创 FPGA 的任意角度畸变矫正：初始方案

> 文档状态：赛前初始方案
>
> 目标平台：紫光同创盘古 50K（MES50HP，PGL50H）
>
> 说明：本文将题面中的要求与工程建议分开。题面要求以官方完整赛题、补充说明和现场规则为准；本文中的分辨率、帧率和模块划分是便于落地的初始假设。

> 开发方式：以 VS Code + Codex 为主，PDS 作为紫光 FPGA 的官方综合、布局布线、位流生成和下载后端。不要把 PDS GUI 当作日常代码编辑器，也不要让 Codex 盲目替代 PDS 的器件/IP/时序流程。

1. 题目理解

从现有题面截图看，本题的核心不是目标检测或 YOLO，而是一个面向真实相机图像的 FPGA 图像校正系统，至少包含三类工作：

1. 解析或获得相机内参矩阵，包括焦距和主点。
2. 估计畸变参数，例如径向畸变参数 `k1`、`k2`，必要时增加 `k3`、切向畸变参数 `p1`、`p2`。
3. 在 FPGA 上对实时图像执行矫正，并用测试图卡、MTF50/SFR、直线度和边缘质量评价结果。

题目中的“任意角度”需要重点确认其准确含义：可能指相机或测试卡以任意角度放置后的透视/旋转矫正，也可能进一步要求镜头径向畸变矫正。初始架构应同时预留旋转、透视和径向映射的接口，但开发顺序先从可控的旋转和径向畸变开始。

1.1 最终目标边界

本文的最终目标不是只完成“固定角度旋转演示”，而是完成一套可以覆盖题目全部任务的系统：

1. 能采集真实摄像头图像，并稳定输出实时视频。
2. 能在 PC 端完成相机标定，得到内参矩阵和畸变参数。
3. 能通过上位机将标定参数、任意角度和矫正模式下发给 FPGA。
4. FPGA 能执行旋转、透视/姿态变化和镜头畸变的组合校正。
5. 支持双线性插值、边界填充和可配置输出窗口。
6. 通过 HDMI 实时显示原图、矫正图和测试叠加结果。
7. 能用测试卡完成矫正前后的直线度、边缘质量、SFR/MTF50 对比。
8. 上位机能保存标定参数、抓取画面、显示指标并生成测试记录。
9. 能给出资源占用、帧率、延迟、误差和不同角度下的稳定性报告。
10. 最终形成可复现的 VS Code 工程、PDS 工程、RTL、模型、上位机、测试数据、视频和设计文档。

因此，本文中的“阶段 1/2/3”只是逐步实现完整系统的顺序，不代表最终方案可以停留在简化模型上。

2. 预期系统形态

推荐的首版系统如下：

```
OV5640 DVP 摄像头
        |
```

```
        v
FPGA 图像采集与 RGB565 解码
        |
        v
DDR3 帧缓存 / 行缓存
        |
        v
坐标反向映射：旋转、透视、径向畸变
        |
        v
最近邻或双线性插值
        |
        +--> HDMI 显示器：实时矫正画面
        |
        +--> HDMI 采集卡：电脑上位机显示/录制
        |
        +--> UART：参数配置、状态、MTF/SFR 结果
```

2.1 摄像头建议

优先选用 OV5640 单目 DVP 摄像头模块。原因是公开资料中已有盘古 50K 配套使用案例，DVP 并行接口比 USB 或 MIPI 更适合 FPGA 初学阶段，且可以从 VGA/640×480 模式开始。

首版建议配置：

- 分辨率：640×480；
- 数据格式：RGB565；
- 输入接口：DVP，包含 PCLK、VSYNC、HREF 和数据线；
- 控制接口：SCCB/I2C，用于设置分辨率、输出格式和曝光；
- 镜头：固定焦距、可手动对焦优先，避免自动对焦导致标定参数变化。

不建议首版使用 USB 摄像头或 MIPI 摄像头。USB 会引入 USB Host/协议处理，MIPI 会引入差分接收和更复杂的物理层/IP，都会偏离本题的核心算法。

2.2 视频输出与电脑连接

FPGA 的实时视频优先走 HDMI：

- 直接接显示器，用于现场演示；
- 接 HDMI USB 采集卡，用于电脑上位机显示、录制和对比。

UART 不承担实时视频传输，只用于下发角度、内参、畸变系数和读取帧率、延迟、MTF/SFR 等结果。若后续确实需要电脑直接接收原始视频，再考虑 UDP/以太网，不作为第一版主线。

2.3 VS Code + Codex 主开发方式

建议把 VS Code 作为唯一日常工作台：编辑 RTL、Python 黄金模型、串口工具、约束、文档和测试脚本；Codex 负责在项目目录内协助拆模块、生成代码、解释报告、编写测试和审查差异；PDS 只负责需要厂商数据库和器件工具的步骤。

推荐的项目目录：

```
.
├── README.md
├── docs/
│   ├── 紫光同创_FPGA_任意角度畸变矫正_初始方案.md
│   ├── protocol.md
│   └── bringup.md
├── rtl/
│   ├── common/
│   ├── camera/
│   ├── video/
│   ├── correction/
│   ├── memory/
│   └── debug/
├── constraints/
│   ├── pins.ucf
│   └── timing.ucf
├── pds/
│   ├── project/          # PDS 工程文件，按实际版本确认是否纳入 Git
│   ├── ip/               # 手工配置或导出的 IP
│   └── scripts/          # PDS 可执行的批处理/Tcl/命令脚本
├── model/
│   ├── calibration.py
│   ├── distortion_model.py
│   ├── fixed_point_model.py
│   └── requirements.txt
├── tests/
│   ├── test_coordinate_map.py
│   ├── test_interpolation.py
│   └── vectors/
├── tools/
│   ├── uart_console.py
│   ├── capture_compare.py
│   └── generate_lut.py
└── .vscode/
    ├── tasks.json
    ├── launch.json
    └── settings.json
```

如果 PDS 工程文件包含绝对路径或版本相关生成内容，应保留一份可复现的工程说明，并把真正的源文件、约束、IP 配置和脚本放在可审查的位置。不要把 `output/`、临时日志、综合缓存和下载生成物当作源代码维护。

VS Code 中的日常循环

```
Codex 提出/修改一个小模块
        ↓
VS Code 查看 diff
        ↓
Python 仿真或定点模型验证
        ↓
PDS 编译/综合/实现
```

```
        ↓
查看资源、时序和错误日志
        ↓
Codex 分析报告并提出下一步修改
```

推荐为 VS Code 配置任务：

- `model:calibrate`：运行棋盘格标定脚本；
- `model:test`：运行坐标映射和插值测试；
- `tools:generate-lut`：生成正余弦或畸变查找表；
- `pds:compile`：调用已验证的 PDS 命令行/批处理入口；
- `pds:reports`：收集综合、布局布线和时序报告；
- `board:uart`：打开串口配置工具；
- `board:capture-compare`：读取 HDMI 采集帧并进行 PC 对比。

如果当前 PDS 版本没有稳定的命令行接口，就把 `pds:compile` 设为“打开工程并提示人工执行”，不要伪造不存在的 CLI。VS Code 仍然可以完成绝大多数源代码、模型和验证工作。

Codex 的推荐职责

适合交给 Codex 的工作：

- 将题目需求拆成 RTL 接口和验收条件；
- 编写 Verilog/SystemVerilog 模块初稿；
- 生成 Python/OpenCV 黄金模型；
- 将浮点模型转换为定点模型；
- 编写边界、溢出、角度和插值测试；
- 分析 PDS 综合/布局布线/时序报告；
- 生成 UART 协议、上位机工具和文档；
- 审查时钟域、复位、流水线延迟和握手信号。

不应直接交给 Codex 自动决定的内容：

- 盘古 50K 的真实引脚和电平标准；
- PDS IP 的最终参数；
- DDR3、HDMI、摄像头的板级连接关系；
- 未经验证的器件原语；
- 是否通过比赛验收。

每次让 Codex 修改 RTL 时，建议使用下面的提示结构：

```
目标：实现 coordinate_mapper 的固定角度版本。
输入：640x480 输出像素坐标，Q16.16 定点参数。
接口：列出每个信号、时钟、复位、有效条件和延迟。
约束：禁止浮点；先不接 DDR3；输出必须可仿真。
验证：增加 0°、90°、负角度、边界和溢出测试。
交付：只修改指定文件，并说明每个时序假设。
```

这样可以让 Codex 围绕你当前的工程状态工作，而不是一次性生成一个无法验证的“大工程”。

3. 算法方案

3.1 软件黄金模型优先

先使用 Python/OpenCV 建立参考结果，输入一张相机拍摄的棋盘格或 SFR 图，完成：

- 相机内参标定；
- 畸变系数估计；
- 旋转/透视变换；
- 径向和切向畸变矫正；
- 最近邻和双线性插值对比；
- 输出矫正图、边缘图、直线度误差和 MTF50/SFR 参考值。

黄金模型必须固定坐标原点、角度正方向、输出画布、边界填充方式和参数单位。否则 FPGA 与 PC 结果不一致时很难定位问题。

3.2 相机标定

相机标定不应在 FPGA 内实时求解。建议使用 PC 离线完成：

1. 打印或购买棋盘格标定板。
2. 从不同位置、不同角度拍摄 10～20 张图。
3. 使用 OpenCV `calibrateCamera` 等方法求取内参矩阵和畸变系数。
4. 将结果转换成定点格式和 FPGA 可读取的参数文件。
5. FPGA 运行时只执行映射和采样。

典型内参矩阵为：

```
K = [ fx  0  cx ]
    [ 0  fy  cy ]
    [ 0   0   1 ]
```

典型畸变模型可以先使用：

```
r2 = x^2 + y^2
x_d = x * (1 + k1*r2 + k2*r2^2) + 2*p1*x*y + p2*(r2 + 2*x^2)
y_d = y * (1 + k1*r2 + k2*r2^2) + p1*(r2 + 2*y^2) + 2*p2*x*y
```

第一版可只支持 `k1`、`k2`，并把 `p1`、`p2` 置零；第二版再增加切向畸变和 `k3`。

3.3 逆向映射

输出端每产生一个像素 `(u_out, v_out)`，就反向计算它在输入图像中对应的位置 `(u_in, v_in)`：

```
输出坐标
   |
   v
去中心化/归一化
   |
   v
旋转或单应性逆变换
   |
   v
径向/切向畸变映射
   |
   v
恢复到像素坐标
   |
   v
从帧缓存取样
```

逆向映射比正向映射更适合视频输出，因为每个输出像素都有明确来源，避免正向投影产生空洞。

3.4 任意角度的实现路线

不要第一版就实现完整浮点三角函数。推荐分三级：

级别 A：预设角度

在 PC 端预先计算 `sinθ`、`cosθ` 和偏移量，FPGA 只读取固定参数。先实现 `-45°`、`0°`、`+45°` 等几个角度。

级别 B：参数化角度

通过 UART 或按键输入角度，FPGA 从 LUT 读取正弦/余弦，或使用 CORDIC 计算。角度范围可先限制在 `-45°～+45°`。

级别 C：组合畸变模型

在角度旋转/透视后加入径向和切向畸变校正，支持从 PC 下发 `fx、fy、cx、cy、k1、k2、p1、p2`。

最终组合模型

最终版本建议统一为一个可配置的逆向映射器，而不是把“旋转模块”和“畸变模块”做成互相独立的临时补丁：

```
输出像素 (u,v)
    ↓
输出坐标归一化
    ↓
逆单应性/姿态变换 H^-1
    ↓
相机畸变模型逆映射
    ↓
输入像素坐标 (x,y)
```

```
    ↓
双线性插值
```

其中：

- `H` 用于表示任意角度、旋转和透视关系；
- `K` 用于相机内参；
- `k1、k2、k3、p1、p2` 用于镜头畸变；
- 参数由 PC 标定/计算后发送到 FPGA；
- FPGA 只负责高吞吐的逐像素映射和采样。

最终不要把矩阵求逆、相机标定和复杂优化放入实时 RTL。矩阵求解、参数拟合和标定属于 PC 侧；FPGA 侧实现确定性的参数化像素处理。

4. FPGA 硬件架构

建议的 RTL 模块如下：

```
top
├── clk_pll
├── ov5640_config
├── camera_capture
├── rgb565_decoder
├── frame_sync
├── ddr3_frame_buffer
├── angle_parameter_ctrl
├── coordinate_mapper
├── fixed_point_math
├── pixel_interpolator
├── boundary_handler
├── hdmi_timing
├── video_overlay
├── uart_config
├── uart_status
└── debug_counter
```

最终版本还应增加以下逻辑能力：

- `transform_mode`：旋转、透视、径向、组合模式；
- `parameter_bank`：双参数组，用于帧边界安全切换参数；
- `frame_status`：帧计数、丢帧、越界、DDR3 欠载和插值异常计数；
- `quality_overlay`：叠加网格、中心点、边界框、当前角度和状态；
- `capture_trigger`：在测试时锁存指定帧，便于与上位机参考图对比。

4.1 `camera_capture`

负责采集 DVP 数据，识别一帧开始/结束和一行有效数据，输出统一的像素流：

```
pixel_data
pixel_valid
line_start
line_end
frame_start
frame_end
```

该模块要处理摄像头时钟域和系统时钟域之间的跨时钟问题，不能直接把异步信号送入后级逻辑。

4.2 ddr3_frame_buffer

任意角度映射会造成非连续读地址，因此完整方案建议预留 DDR3 帧缓存。第一版可以采用单帧写入、显示端读取；稳定后再升级为乒乓缓冲。

需要验证：

- DDR3 IP 是否由当前 PDS 版本提供；
- 行突发读写能否满足像素吞吐量；
- 写入时钟和读取时钟是否独立；
- 帧切换时是否出现撕裂；
- 输入输出分辨率和像素格式。

如果 DDR3 调试成本过高，可先用 ROM/片上 RAM 读取测试图片，先验证坐标映射和插值模块。

4.3 coordinate_mapper

该模块是核心。它接收输出像素坐标和参数，输出：

```
input_x_integer
input_y_integer
input_x_fraction
input_y_fraction
input_valid
```

建议使用 Q 格式定点数，例如 Q12.20 或 Q16.16。具体格式根据分辨率、参数范围和乘法器资源确定。第一版不要使用浮点 RTL。

4.4 pixel_interpolator

实现顺序：

1. 最近邻插值：只读取一个像素，最快速、最容易验证。
2. 双线性插值：读取四个邻域像素，画质明显更好。

双线性插值需要正确处理 RGB565 的三个颜色分量，建议先拆成 R/G/B 后计算，再重新打包。

4.5 boundary_handler

输入坐标超出图像范围时，统一处理为：

- 填充黑色；或
- 复制边缘像素。

初始方案建议填充黑色，逻辑最简单，且便于展示矫正区域边界。

最终版本应至少支持两种边界模式：黑色填充和边缘复制，并允许通过上位机切换。对越界像素必须有计数器，否则无法解释不同角度下画面边缘的黑区变化。

5. 测试卡与评价方案

5.1 低成本赛前方案

第一阶段不必购买昂贵专业卡，可以使用：

- A3/A4 棋盘格标定图；
- 黑白网格线；
- 自制倾斜黑色矩形/斜边图；
- 打印后贴到平整硬纸板。

它们足以完成标定流程、直线弯曲观察和 FPGA 前后效果对比。

5.2 专业图卡

如果最终需要更规范地报告 MTF50/SFR，再购买 ISO 12233/eSFR 图卡。专业图卡价格可能从数百元到数千元不等，首版不应把它作为项目启动前提。

5.3 验证指标

建议至少记录：

- 输入和输出分辨率；
- 实际帧率；
- 单帧延迟；
- FPGA 资源占用；
- 最大时钟频率；
- 矫正前后直线度误差；
- 矫正前后 MTF50/SFR；
- 边缘是否出现明显拉伸、断裂或黑洞；
- 不同角度下的稳定性。

5.4 完整测试矩阵

最终验收不能只测一个角度和一张图，至少应建立如下矩阵：

类别	测试内容
标定	不同距离、不同姿态、不同图卡位置下的内参重投影误差
角度	0°、正角度、负角度、大角度以及角度切换

类别	测试内容
畸变	仅径向、径向+切向、透视、组合模型
画质	最近邻与双线性插值，中心与边缘区域，水平与垂直方向
实时性	分辨率、帧率、处理延迟、持续运行时间
健壮性	参数越界、输入丢帧、DDR3 欠载、摄像头重新上电
对比	PC 浮点参考、PC 定点参考、FPGA 输出三者的像素误差

最终报告应给出 FPGA 与 PC 黄金模型之间的最大绝对误差、平均绝对误差和可接受误差阈值，而不是只展示一张肉眼看起来正常的图片。

MTF50/SFR 的完整计算可以先放在 PC 上完成：FPGA 输出矫正画面，HDMI 采集卡或抓帧方式送到 PC，再用 Python/OpenCV 或专用工具分析。FPGA 内部只需回传必要统计值即可。

6. 上位机方案

6.1 第一版

使用串口工具或 Python 小程序完成：

- 设置角度；
- 设置是否启用畸变矫正；
- 下发 `fx、fy、cx、cy、k1、k2`；
- 读取帧率、坐标、状态寄存器。

6.2 第二版

使用 HDMI 采集卡将实时画面输入 PC，上位机显示：

- 原始画面；
- FPGA 矫正画面；
- 角度和参数；
- MTF50/SFR 测试结果；
- 帧率和延迟。

6.3 最终上位机功能

建议使用 Python + PySide6/PyQt + OpenCV，在 VS Code 中维护。最终上位机至少包含：

- 摄像头/板卡连接状态；
- 标定图片导入和标定参数生成；
- 内参矩阵、畸变参数和重投影误差显示；
- 角度、输出分辨率、插值和边界模式设置；
- 参数版本保存与加载；
- UART 参数下发和 ACK/CRC 校验；
- HDMI 采集画面显示；
- 原图、矫正图和差分图对比；
- SFR/MTF50、直线度、帧率和延迟统计；

- CSV/JSON/PNG/PDF 或 Markdown 测试记录导出。

6.4 通信协议初稿

建议使用简单定长帧，避免初期引入复杂协议：

```
帧头  命令  长度  数据  CRC16
0x55  1B    1B    N字节  2B
```

首批命令可以包括：

命令	用途
0x01	设置角度
0x02	设置内参
0x03	设置畸变系数
0x04	开关矫正
0x81	读取运行状态
0x82	读取帧率和错误计数

字段宽度和缩放比例在 RTL 与 PC 端确认后再固化，不要在未验证前写死为最终协议。

7. 分阶段开发计划

阶段 0：环境和资料确认

- 固定 PDS 版本；
- 获取盘古 50K 原理图、引脚约束、DDR3/HDMI/摄像头例程；
- 确认 OV5640 模块接口和电平；
- 确认完整题面对“任意角度”的数学含义；
- 建立 Git 仓库和文档目录。

验收：官方点灯、摄像头和 HDMI 示例至少有一个可以独立运行。

阶段 1：视频链路

- 完成摄像头寄存器配置；
- 采集 RGB565；
- HDMI 输出彩条；
- 摄像头画面直通 HDMI；
- 记录同步和帧率。

验收：显示器稳定显示摄像头实时画面 10 分钟以上。

阶段 2：PC 黄金模型

- 完成棋盘格标定；
- 保存内参和畸变参数；
- 实现软件逆向映射；
- 生成 FPGA 定点参数；
- 建立测试图片和误差对比脚本。

验收：软件模型的矫正直线和参考结果稳定。

阶段 3：FPGA 固定参数映射

- 用 ROM 图片验证坐标映射；
- 实现固定角度旋转；
- 实现最近邻插值；
- 再接入 DDR3 帧缓存；
- 通过 HDMI 显示矫正图。

验收：固定角度下 FPGA 输出与软件参考图大体一致，边界行为正确。

阶段 4：参数化和画质提升

- UART 下发角度；
- LUT 或 CORDIC 计算正余弦；
- 增加双线性插值；
- 增加 `k1`、`k2` 径向畸变；
- 增加参数保存和恢复。

验收：多个角度下能稳定运行，且画质明显优于最近邻版本。

阶段 5：指标和演示

- 完成 SFR/MTF50 参考测试；
- 测量直线度误差；
- 统计帧率、延迟和资源占用；
- 制作原图/矫正图/指标对比；
- 准备演示视频和设计文档。

验收：形成可重复演示的完整闭环。

阶段 6：完整任务闭环

这一阶段是最终交付阶段，不应省略：

- 将旋转、透视和径向/切向畸变统一到组合映射器；
- 完成任意角度参数化和安全的帧边界切换；
- 完成双线性插值和边界模式切换；
- 完成 PC 标定、参数生成、下发、回读和版本管理；
- 完成 HDMI 采集、PC 参考图和 FPGA 输出图的自动化对比；
- 完成全测试矩阵和数据记录；
- 优化 DDR3 带宽、DSP 使用、流水线和时序；
- 完成连续运行、异常恢复和现场重启测试。

验收：题目中列出的每一项功能都有可操作入口、可观察输出和可保存的测试证据。

阶段 7：比赛交付物

- VS Code 工程和安装说明；
- PDS 工程、版本和器件配置说明；
- RTL 源码及模块接口文档；
- Python 标定、黄金模型和自动化测试代码；
- 上位机程序及通信协议；
- 引脚约束、时序约束和 IP 配置记录；
- 资源、时序、帧率和延迟报告；
- MTF50/SFR 和直线度测试数据；
- 演示视频和现场操作手册；
- 已知限制、复现步骤和故障排查表。

验收：换一台同配置电脑后，按照 README 能完成模型测试、PDS 构建、下载和演示。

8. 风险与应对

风险	影响	应对
题目中的“任意角度”定义不清	算法方向错误	尽快向官方技术群确认数学模型和验收方式
OV5640 图像链路不通	无法开始算法演示	先跑官方例程，固定使用 VGA/RGB565
DDR3 随机读效率不足	帧率低或画面撕裂	降低分辨率，采用行缓存/乒乓缓冲/突发读取
定点溢出	图像严重错位	PC 端先做定点仿真，增加边界和溢出测试
双线性插值资源过高	时序不收敛	先最近邻，乘法分时或利用 DSP/硬核资源
标定参数不稳定	结果不可重复	固定焦距、曝光和测试距离，保存参数版本
只在 PC 上验证	FPGA 现场效果不可信	早期就接入真实摄像头和 HDMI

9. 初始完成标准

第一版里程碑可以定义为：

1. OV5640 能稳定采集 640×480 RGB565 图像。
2. FPGA 能通过 HDMI 显示原始画面。
3. PC 能通过棋盘格完成一次内参和畸变参数标定。
4. FPGA 能使用固定参数完成至少一个角度的图像矫正。
5. FPGA 画面与软件黄金模型在坐标、边界和直线形态上基本一致。
6. 能通过 UART 修改角度或开关矫正。
7. 能展示矫正前后图像，并记录帧率、延迟和简单质量指标。

但最终完成标准还必须增加：

8. 支持组合变换，而不是只支持固定角度旋转。
9. 支持完整的内参和畸变参数链路。
10. 支持双线性插值和至少两种边界处理方式。

11. 支持上位机标定、参数管理、实时配置和测试报告生成。
12. 完成多角度、多图卡、多分辨率和长时间稳定性测试。
13. 给出 MTF50/SFR、直线度、像素误差、资源占用、帧率和延迟的可复现数据。
14. 完成最终视频、源码、文档和 PDS 工程交付。

10. 当前最应该做的三件事

1. 获取并跑通盘古 50K 的 OV5640 摄像头直通 HDMI 例程。
2. 用 OpenCV 对同一摄像头完成棋盘格标定，保存 `K`、`k1`、`k2`。
3. 用一张静态测试图先实现 Python 逆向映射和定点仿真，再开始写 FPGA 坐标映射模块。

开发顺序上仍然应先把“真实摄像头输入—FPGA 输出—软件参数—HDMI 显示”的最小闭环跑通，但最终不能停在这个原型阶段。后续必须回到组合映射、完整参数链路、双线性插值、指标测试和交付物闭环，直到题目要求逐项有证据覆盖。

参考资料

- 紫光同创 PDS 官方产品介绍：https://pangomicro.com/product/pds/
- 盘古 50K + OV5640 公开使用案例：https://bbs.elecfans.com/jishu_2362502_1_1.html
- SFR/MTF 测试卡价格与规格参考：https://rdbuy.com/search/search?search=SFR

附录 A：PDS Lite + VS Code + Codex 工具链探索进度

> **结论：**可以采用 VS Code + Codex 作为日常开发入口，PDS Lite 作为紫光 FPGA 的综合、实现、位流生成和下载后端。当前已经验证 PDS Lite 的命令行工具入口，但尚未用本题真实工程完成端到端构建，因此后续仍需用首次建立的真实工程做一次闭环验证。

A.1 本机已验证环境

项目	验证结果
PDS Lite	`Pango Design Suite 2022.2-SP1-Lite`，安装目录为 `D:\PDS_lite\PDS_2022.2-SP1-Lite`。
VS Code	版本 `1.139.1`，`code` 命令可以从终端调用。
盘古 50K 支持文件	安装包中存在 `pgl50h` 架构目录和 `Logos-PGL50H.ini`；最终器件、封装和速度等级仍以开发板资料为准。
PDS 主流程	`pds_shell.exe` 已启动并输出帮助，支持工程、脚本、工作目录和 action 参数。
配置/下载	`cdt_cfg.exe` 与 `cdt_cfg_shell.exe` 已启动并输出帮助，支持 Tcl 脚本流程。
IP 工具	`ip_compiler.exe` 提供命令行入口；IP 参数配置和生成仍建议先用图形界面。

A.2 可以交给 Codex 的工作

- 编辑 Verilog/SystemVerilog、摄像头接口、DDR3、坐标映射、插值和 HDMI 模块。
- 编写和维护 PDS Tcl、Windows 批处理、VS Code `tasks.json` 以及约束文件。
- 建立 Python/OpenCV 黄金模型、定点模型、标定工具和自动化测试。
- 读取综合、布局布线、时序和资源报告，定位错误并提出 RTL 修改。
- 编写 UART 参数协议、上位机工具、测试数据处理和项目文档。

A.3 仍需要人工参与的环节

- 第一次新建 PDS 工程、选择正确器件/封装、确认开发板管脚。
- 通过 PDS/IP Compiler 图形界面完成首次 IP 配置。
- 插拔 USB/JTAG、连接摄像头和显示器，并确认板级电气连接。
- 观察显示器上的实时画面，判断摄像头曝光、同步、撕裂和实际画质。

A.4 已确认的命令行接口

`pds_shell.exe -help` 输出的主要参数如下：

```
-file string                指定工程脚本文件，以批处理模式运行
-project string             指定工程文件，以批处理模式运行
-project_file_path string   指定工程文件路径
-work_dir string            指定工作目录
-project_name string        指定输出工程名称
-run string                 指定需要运行的 action
-continue                   脚本出错后继续执行
-version                    输出版本信息
```

官方 PDS 用户手册列出的主要 action 包括 `compile`、`dev_map`、`pnr`、`gen_bit_stream`、`gen_netlist` 和 `report_timing`。官方用户手册还支持通过 `File -> Export Project To Tcl` 导出 Tcl，并生成 Windows `.bat` 文件。

`cdt_cfg.exe`/`cdt_cfg_shell.exe` 可以执行配置工具 Tcl，官方配置手册确认了 `-file`、`-args`、`-load_dir` 和 `-report_dir` 等参数。实际下载前仍需确认 USB/JTAG 驱动、板卡连接和目标器件识别状态。

A.5 推荐的 VS Code 工程闭环

```
Codex 修改 RTL / 约束 / Python 模型
          ↓
VS Code 查看 diff 并运行模型测试
          ↓
PDS shell 执行综合、映射、PnR、时序和位流生成
          ↓
Codex 分析报告并修正问题
          ↓
cdt_cfg 执行下载脚本
          ↓
摄像头 + HDMI + 板卡现场验证
```

建议首次手动建立 PGL50H 工程并导出 Tcl，之后再把脚本纳入项目：

```
$PDS_ROOT = 'D:\PDS_lite\PDS_2022.2-SP1-Lite\bin'
& "$PDS_ROOT\pds_shell.exe" -file .\pds\scripts\build.tcl -work_dir .\build\pds
& "$PDS_ROOT\pds_shell.exe" -project .\pds\project\main.pds -run pnr -work_dir .\build\pds
& "$PDS_ROOT\cdt_cfg.exe" -file .\pds\scripts\download.tcl
```

> **当前状态：**命令行入口和官方脚本能力已经确认；真实工程的首次新建、器件/封装选择、IP 参数和 PDS 端到端构建仍是下一步验证项。完成这一步后，Codex 可以接管大部分 RTL、模型、脚本和报告分析工作，但不会替代板卡连接和最终硬件验收。

A.6 下一步

1. 在 PDS Lite 中新建 PGL50H 工程，确认开发板实际封装和速度等级。
2. 导入一个最小 Verilog 顶层，运行一次综合并导出 Tcl/.bat。
3. 将工程放入英文路径，例如 `D:\fpga\pgl50k_distortion`，再由 VS Code + Codex 建立自动构建任务。
4. 先跑通 OV5640 直通 HDMI，再接入畸变矫正坐标映射。